
B SOS

Community emergency response platform

Product, technical and delivery plan

Version 1.0 - September 2026

Full-stack course capstone, built as a production-grade product

This document describes what B SOS is, how it will be built, in which order, how the work is split between the two developers, and what makes the finished product worth its price. It is designed to be followed step by step from an empty repository to a deployed, demonstrable system.

1. Product overview

The problem

When something goes wrong on the road or at home - an accident, a flat tyre, a sudden medical situation, someone stranded without transport - the official emergency services are often slow to arrive, and the person has no way to reach the nearest people who could actually help. Meanwhile, within a few hundred meters there is usually a nurse, a mechanic, or simply a neighbour with a car who would help immediately if they knew.

The solution

B SOS connects a person in need with the closest verified helpers in real time. One tap publishes an SOS with its type and location; the platform notifies the nearest matching helpers, the first to accept is guided to the location, and the requester sees them approaching live on a map. B SOS complements the official emergency services - it never replaces them. For life-threatening situations the app always shows the emergency number first.

Who uses it

Role	What they do	Notes
Requester	Publishes an SOS, sees the helper approach, rates the help afterwards	Any registered user
Helper	Declares skills (medical, mechanic, driver, general), receives nearby SOS alerts, accepts and responds	Any user can also be a helper; medical helpers are verified
Verified professional	Doctor / nurse / paramedic / mechanic with uploaded credentials	Approved by admin, badge shown in app, priority routing
Admin	Approves credentials, reviews reports, bans abusers, sees analytics	Internal dashboard

Guiding principles

- Speed first: from tap to notified helpers in under two seconds.
- Trust is a feature: verification, ratings, and abuse prevention are core, not extras.
- Works on a weak connection: the app must degrade gracefully with poor mobile data.
- Privacy by default: exact locations are shared only with the accepted helper and only while the SOS is active.
- Always a fallback: the emergency number and a plain phone call are always one tap away.

2. Feature scope

Features are grouped into four tiers. Build them in this order. Each tier leaves the product in a working, demonstrable state, so if time runs out the project is still complete.

Tier 1 - Core MVP (must ship)

- Register / login (email + password, JWT, refresh tokens). Phone number required and verified by SMS code.
- Profile: name, photo, phone, helper skills (multi-select), availability toggle ("I can help now").
- Create SOS: type (Medical / Vehicle / Transport / Safety / Other), short description, optional photo, auto-captured location with manual pin adjustment.
- Nearby matching: server finds helpers within a radius (start 3 km, expand to 5 and 10 km if no response) whose skills match the SOS type.
- Real-time notifications to matched helpers (WebSocket while app is open, push when closed).
- Accept flow: first helper to accept is assigned; others are told it is taken. Requester sees helper name, photo, rating, ETA.
- Live tracking: helper location streamed to requester until arrival.
- SOS lifecycle: OPEN -> ACCEPTED -> EN_ROUTE -> ARRIVED -> RESOLVED / CANCELLED, with timestamps for each step.
- Emergency number always visible on the SOS screen.
- Admin dashboard v1: list users, list SOS, ban user.

Tier 2 - Trust and safety

- Two-way rating after every resolved SOS (1-5 stars + optional comment).
- Credential verification: upload document, admin approves, verified badge, priority in matching for medical SOS.
- Report and block: report a user or an SOS; three reports auto-suspends until admin review.
- Anti-abuse: rate limit SOS creation (max 3 open per user per day), require phone verification, flag users with high cancel rate.
- Safety check-in: requester gets an "Are you okay?" prompt after the helper arrives; a trusted contact can be notified automatically when an SOS is created.
- In-app chat between requester and assigned helper (text only, stored, closes when SOS resolves).

Tier 3 - Product quality

- Push notifications (Firebase Cloud Messaging) so helpers are reached even with the app closed.
- Progressive Web App: installable on phone, offline shell, background location permission handling.
- Smart escalation: if nobody accepts within 60 seconds, widen radius and re-notify; after 3 minutes, suggest calling emergency services.
- SOS history for users; helper statistics (responses, average arrival time, rating).
- Multi-language UI (Arabic RTL + English) with a language switch.
- Admin analytics: SOS heatmap, response times, active helpers by area, top helpers.

Tier 4 - Growth and premium (post-course roadmap)

- Organisations: a taxi company, a clinic, or a university registers its staff as a helper group with its own dashboard.

-
- Community zones: neighbourhood groups that receive local alerts (power cuts, road closures) in addition to SOS.
 - Native mobile apps (React Native) reusing the same API.
 - Helper incentives: points, badges, and optional tipping through a payment provider.
 - Integration with insurance / roadside assistance companies as a B2B channel.

3. Technical architecture

Stack

Layer	Technology	Why
Frontend	React + Vite, React Router, Tailwind CSS, Leaflet (maps), Socket.io-client	Fast to build, what you already learned, Leaflet is free with no API key
Backend	Node.js + Express, Socket.io, JWT auth, Zod validation	Same language as frontend, mature real-time support
Database	PostgreSQL + PostGIS (geo queries) via Prisma ORM	Real distance queries in SQL, indexes on location, strong relations
Cache / queue	Redis (online helpers, live positions, rate limits, job queue with BullMQ)	Live positions change every few seconds; never write them to Postgres each time
Files	Cloudinary or S3-compatible bucket for photos and credential documents	Never store files in the database
Push	Firebase Cloud Messaging (web push)	Free, reliable, works with PWA
SMS	Twilio or a local provider for phone verification	Trust starts with a real phone number
Hosting	Frontend: Vercel. Backend + Redis + Postgres: Railway or Render. Domain + HTTPS	Free tiers to start, one-click deploys from GitHub
Dev tooling	GitHub, ESLint + Prettier, Vitest (unit), Supertest (API), Playwright (e2e), GitHub Actions CI	Professional workflow; CI runs tests on every pull request

How a request flows

- Requester taps SOS -> client sends POST /sos with type, text, lat/lng, accuracy.
- API validates, stores the SOS in Postgres with a PostGIS point, sets status OPEN.
- Matching service queries: helpers within radius R, available = true, skills overlap SOS type, not banned, not already on an active SOS. Ordered by distance, verified first for medical.
- For each matched helper: emit a Socket.io event to their room if connected; otherwise send an FCM push. Store the notification.
- A helper taps Accept -> POST /sos/:id/accept. Done inside a database transaction with a row lock so only one accept can win. Others receive sos:taken.
- Helper's client streams location every 3-5 seconds via socket -> stored in Redis -> forwarded to the requester's room only.
- On RESOLVED, final positions and timings are written to Postgres for history and analytics; Redis entries are cleared.

Repository layout

One GitHub monorepo with two folders: **/client** (React) and **/server** (Express). Shared types in **/shared**. One README with setup instructions, one .env.example per side. Branch per feature, pull request with

review by the other partner before merging to main.

4. Data model

Start with these tables. Every table has id (uuid), created_at, updated_at.

Table	Key columns	Notes
users	email, password_hash, name, phone, phone_verified, photo_url, role (USER/ADMIN), is_banned, rating_avg, rating_count, trusted_contact_phone	One row per person
helper_profiles	user_id, skills[] (MEDICAL, MECHANIC, DRIVER, GENERAL), is_available, verification_status (NONE/PENDING/VERIFIED/REJECTED), credential_doc_url, last_location (geography Point), last_seen_at	GIST index on last_location
sos_requests	requester_id, type, description, photo_url, location (geography Point), accuracy_m, status, helper_id, accepted_at, arrived_at, resolved_at, cancelled_reason	Index on status + location
sos_notifications	sos_id, helper_id, channel (SOCKET/PUSH), sent_at, seen_at, responded (ACCEPT/DECLINE/NONE)	Powers analytics and escalation
location_updates	sos_id, user_id, location, recorded_at	Only final samples; live stream lives in Redis
messages	sos_id, sender_id, body, sent_at, read_at	Chat, closed when SOS resolves
ratings	sos_id, from_user_id, to_user_id, stars, comment	Unique (sos_id, from_user_id)
reports	reporter_id, reported_user_id, sos_id, reason, status, admin_note	Three open reports = auto-suspend
admin_actions	admin_id, action, target_user_id, note	Audit log

Key SQL: nearest available helpers

```
SELECT u.id, u.name, u.rating_avg, hp.verification_status,
       ST_Distance(hp.last_location, ST_MakePoint($lng,$lat)::geography) AS meters
FROM helper_profiles hp JOIN users u ON u.id = hp.user_id
WHERE hp.is_available AND NOT u.is_banned
      AND hp.skills && $skills
      AND hp.last_seen_at > now() - interval '10 minutes'
      AND ST_DWithin(hp.last_location, ST_MakePoint($lng,$lat)::geography, $radius)
ORDER BY (hp.verification_status = 'VERIFIED') DESC, meters ASC
LIMIT 15;
```

5. API and real-time events

REST endpoints

Method + path	Purpose
POST /auth/register, /auth/login, /auth/refresh, /auth/verify-phone	Accounts
GET/PATCH /me	Profile, skills, availability, trusted contact
POST /me/credentials	Upload document for verification
POST /sos	Create SOS (returns id, triggers matching)
GET /sos/:id	Details including assigned helper
POST /sos/:id/accept /decline /arrived /resolve /cancel	Lifecycle transitions (server validates allowed transitions)
GET /sos/mine	History as requester and as helper
POST /sos/:id/messages, GET /sos/:id/messages	Chat
POST /sos/:id/rating	Rate the other party
POST /reports	Report a user
GET /admin/users, PATCH /admin/users/:id/ban, GET /admin/credentials, PATCH /admin/credentials/:id, GET /admin/stats	Admin

Socket.io events

Event	Direction	Payload
helper:location	client -> server	{lat, lng, accuracy} every 3-5 s while available or on an SOS
sos:new	server -> matched helpers	{sosId, type, distance, requesterRating}
sos:taken	server -> other helpers	{sosId}
sos:accepted	server -> requester	{helper: {name, photo, rating}, etaMinutes}
sos:helper-location	server -> requester	{lat, lng} live stream
sos:status	server -> both	{sosId, status}
chat:message	both	{sosId, body, senderId, sentAt}

Every socket connection authenticates with the JWT on connect and joins two rooms: user:{id} and, while assigned, sos:{id}. Location events are never broadcast to everyone - only to the specific room that is allowed to see them.

6. Location accuracy and response time

The prototype proved that browser geolocation works between two devices on different networks. It also showed that laptop positions can be hundreds of meters off. These are the improvements, in order of impact.

Get a better position

- Always request **enableHighAccuracy: true** and use **watchPosition**, not a single **getCurrentPosition**. The first fix is rough; the second and third are usually much better. Keep the best (lowest accuracy value) of the first 5 seconds.
- Store and display the **accuracy** value with every position. If it is above 100 m, show the pin with a wide circle and ask the user to drag it to the right spot before sending.
- On phones use GPS (automatic when high accuracy is requested). Laptops have no GPS; treat any laptop position as approximate.
- Discard obvious jumps: if a new sample is more than 200 m from the previous one but arrived less than 2 seconds later, ignore it.
- Add a reverse-geocoded street name (Nominatim, free) next to the pin so the requester can confirm "yes, that is my street".
- Google Maps Geolocation API is an option later: it is slightly better indoors thanks to Google's Wi-Fi database, but costs money and needs a billing account. Leaflet + device GPS is enough for the course version.

Make it faster

- Ask for location permission once, at onboarding, so the SOS button does not wait for a permission dialog.
- Keep a warm position in memory: when the app is open, **watchPosition** runs quietly, so tapping SOS sends the last known good position instantly and refines it afterwards.
- Send the SOS immediately with whatever accuracy you have, then send location updates; do not block the emergency on a perfect fix.
- Matching must be one indexed PostGIS query (< 20 ms) plus fan-out through Redis. Target: helpers notified within 2 seconds of the tap.
- Live helper positions go to Redis and the socket only; Postgres is written at the end. This keeps the tracking smooth and the database small.

Measure it

Log these for every SOS and show them in the admin dashboard: time to first notification, time to first accept, helper distance at accept, helper arrival time, requester accuracy at creation. These numbers are what you show a client to prove the product works.

7. Delivery roadmap (10 weeks)

Assumes two developers working part-time. Each week ends with a working demo. Adjust to your actual deadline by merging or dropping the later weeks.

Week	Goal	Developer A (backend lead)	Developer B (frontend lead)
1	Foundations	Repo, Express skeleton, Postgres + PostGIS on Railway, Prisma schema, auth endpoints, CI	Vite + React + Tailwind, routing, auth pages, API client, design tokens and component library
2	Profiles & helpers	Profile and helper endpoints, skills, availability, location upsert, nearest-helpers query	Profile screens, skills selector, availability toggle, permission onboarding flow
3	SOS creation	POST /sos, validation, matching service, notification table	SOS screen: type picker, map with draggable pin, accuracy circle, emergency number
4	Real-time	Socket.io auth, rooms, sos:new fan-out, Redis for live positions	Socket client, helper alert UI, accept / decline, requester waiting screen
5	Tracking & lifecycle	Accept transaction with row lock, status transitions, helper-location relay, resolve	Live map with helper marker and ETA, status timeline, resolve and cancel flows
6	Trust	Ratings, reports, auto-suspend, credential upload and admin approval	Rating screen, report dialog, verified badge, credential upload UI
7	Chat & push	Messages endpoints and socket events, FCM push sending, escalation job (BullMQ)	Chat UI, push permission and service worker, PWA manifest, installable
8	Admin	Stats endpoints, audit log, ban and suspend	Admin dashboard: users, SOS list, credentials queue, heatmap and response-time charts
9	Quality	Unit + API tests, load test matching with 5,000 fake helpers, security review (rate limits, input validation, CORS, helmet)	E2E tests with Playwright, Arabic RTL and English, accessibility pass, mobile polish
10	Launch	Production deploy, backups, monitoring (Sentry + uptime), README and API docs	Landing page, demo video, presentation deck, seed data for the live demo

Definition of done for every feature

- Works on a phone over mobile data, not only on localhost.
- Has at least one automated test.
- Reviewed by the other partner in a pull request.
- Errors are handled and shown to the user in plain language.

8. Security, privacy and operations

Security checklist

- Passwords hashed with bcrypt (cost 12). JWT access token 15 min, refresh token 7 days in httpOnly cookie.
- All input validated with Zod on the server; never trust client-side checks.
- Rate limits: login 5/min per IP, SOS creation 3 open per user, location updates 1 per second.
- helmet, CORS restricted to the frontend domain, HTTPS everywhere.
- Role checks in middleware; admin routes on a separate router.
- File uploads: type and size checked, stored outside the server, credential documents visible only to admins.
- Secrets only in environment variables; .env never committed; rotate any key that was ever pasted anywhere.

Privacy rules

- Exact requester location is shown only to the helper who accepted, and only while the SOS is active.
- Helper live location is shown only to the requester of the SOS they accepted.
- Location history is kept for 30 days for safety investigations, then deleted.
- Users can delete their account; SOS records are anonymised, not removed, to keep statistics honest.

Testing strategy

Level	Tool	What to cover
Unit	Vitest	Matching rules, status transitions, distance helpers, escalation timing
API	Supertest	Every endpoint: happy path, validation errors, auth errors, double-accept race
Real-time	Socket.io client in tests	Room isolation: a helper must never receive another SOS's positions
E2E	Playwright	Two browser contexts: one requester, one helper, full SOS to resolution
Load	k6 or autocannon	500 concurrent helpers streaming location; matching under 100 ms

Operations

- Sentry for error tracking on both client and server.
- Uptime monitor pinging /health every minute.
- Daily Postgres backups (Railway / Render provide this) and a tested restore.
- Structured logs (pino) with the sosId on every line so an incident can be traced end to end.

9. Why B SOS is worth 50,000 ILS

A client paying this amount is buying a working, deployable system with real-time infrastructure, a trust layer, an admin back-office, and a clear growth path - not a school demo. This is what the price covers.

What is delivered	Business value
Real-time matching and live tracking engine (Socket.io + Redis + PostGIS)	The hard technical core; most competitors' apps are just forms and phone numbers
Verified helper network with credentials, ratings and abuse controls	Trust is what makes people actually use an emergency app
Admin dashboard with analytics and heatmaps	Lets a municipality, insurer or company run the service and prove impact
Installable PWA with push notifications, Arabic + English	No app-store delay, reaches helpers with the app closed, fits the local market
Production deployment with monitoring, backups, CI and tests	It keeps working after the handover
Documentation: API docs, README, data model, this plan	A second team can maintain and extend it
Organisation and B2B roadmap (Tier 4)	Path to recurring revenue: clinics, taxi fleets, insurers, universities

Comparable market signals

Roadside-assistance and community-safety apps are typically licensed to companies for tens of thousands of dollars per year, and custom real-time platforms with an admin back-office are routinely quoted at 40,000-80,000 USD by agencies. Delivering B SOS at 50,000 ILS with this scope is competitive, and the organisation features in Tier 4 create ongoing revenue beyond the initial build.

How to present it

- Open the demo with a live two-phone SOS: tap, notification arrives on the second phone within two seconds, accept, live marker moves, resolve, rate.
- Then show the admin dashboard: the SOS you just did appears in the heatmap and the response-time chart.
- End with the roadmap slide: organisations, native apps, B2B - the story of where the product goes next.

10. Immediate next steps

- Create the GitHub repo with /client and /server, add this PDF to /docs.
- Create the Railway project: Postgres with PostGIS enabled, Redis.
- Developer A: Prisma schema from section 4 and auth endpoints. Developer B: Vite app, auth pages, design tokens.
- Book a 30-minute sync twice a week: demo what works, agree next tasks, review each other's pull requests.

-
- Keep the prototype you built this week as a reference; the real app reuses its ideas, not its code.